

Что такое видео

сигнал, цвет, сжатие, кодеки

Александр Савельев

Курс: Обработка видео в контексте машинного обучения и ИИ
Лекция 1

План лекции

1. Зачем отдельный курс про видео
2. Видео как сигнал: дискретизация и теорема Найквиста
3. Временной алиасинг
4. Сколько «весит» несжатое видео
5. Цвет: почему не RGB, а YCbCr
6. Chroma subsampling
7. Intra-кодирование: DCT и квантование
8. Inter-кодирование: компенсация движения
9. Типы кадров и структура GOP
10. Кодеки и битрейты
11. Следствия для ML

1. Зачем отдельный курс про видео

Картинка — это функция $I(x, y)$. Видео — функция $I(x, y, t)$. Добавление одной переменной меняет всё.

Что даёт ось t :

- ▶ Движение и траектории
- ▶ Контекст соседних кадров
- ▶ Избыточность (99% похоже) — можно жать
- ▶ Новые задачи: трекинг, действия, генерация

Что это стоит:

- ▶ Объём: $\times 25$ –60 на секунду
- ▶ Декодирование: часто дороже модели
- ▶ Разметка: минуты $\rightarrow$ часы
- ▶ Модели: 3D-свёртки, квадратичное внимание

Идея всего курса: каждый метод — ответ на два вопроса — как моделировать ось t и как использовать избыточность соседних кадров.

2. Видео как сигнал

Определение: видео

Видео — это дискретизация непрерывной функции

$$I: \mathbb{R}^2 \times \mathbb{R} \rightarrow \mathbb{R}^3, \quad (x, y, t) \mapsto (R, G, B)$$

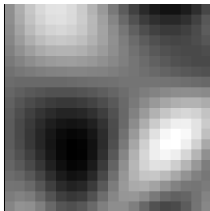
по пространству (с шагом $\Delta x, \Delta y$) и по времени (с шагом $\Delta t = 1/\text{fps}$).

Дискретный тензор: $X[t, i, j, c] = I(j\Delta x, i\Delta y, t\Delta t)$, где t — номер кадра, i, j — строка и столбец пикселя, c — канал (R/G/B), $\text{fps} = 1/\Delta t$ (типично 24, 25, 30, 60).

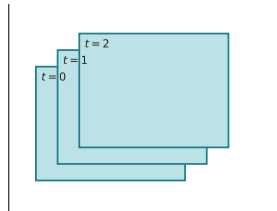
Непрерывная сцена
 $I(x, y, t)$



Дискретизация по пространству
пиксели $X[i, j]$



Дискретизация по времени
кадры $X[t, i, j]$



2. Частота сигнала: формальное определение

Периодический сигнал и его частота

Непрерывный сигнал $x(t)$ называется *периодическим* с периодом $T > 0$, если для всех t :

$$x(t + T) = x(t).$$

Частота — число повторений периода за секунду:

$$f = \frac{1}{T} \quad [\text{Гц}], \quad \omega = 2\pi f \quad [\text{рад/с}].$$

Канонический пример — гармоника $x(t) = \sin(2\pi ft)$. На колесе, вращающемся со скоростью f об/с, вертикальная координата точки — именно такая гармоника.

Свойство Фурье. Любой достаточно хороший сигнал раскладывается в сумму гармоник $\sin(2\pi f_k t)$. Поэтому вопрос «какая частота в сцене» сводится к вопросу о *старшей гармонике* $f_{\max}$ в её разложении.

2. Ограничение дискретизации: теорема Найквиста

Теорема Котельникова-Найквиста

Сигнал $x(t)$ с ограниченным спектром (все частоты $\leq f_{\max}$) однозначно восстанавливается по выборкам $x(k/f_s)$, $k \in \mathbb{Z}$, тогда и только тогда, когда

$$f_s > 2f_{\max}.$$

Порог $2f_{\max}$ называют *частотой Найквиста*. Восстановление — по формуле sinc-интерполяции.

В контексте видео: $f_s = \text{fps}$. Если движение в сцене быстрее половины fps, восстановить его нельзя — оно превратится в артефакт (алиасинг).

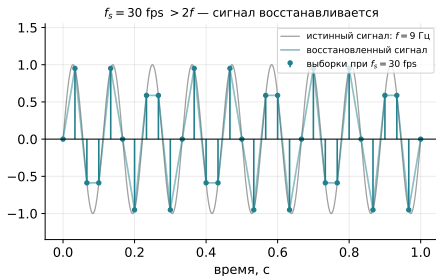
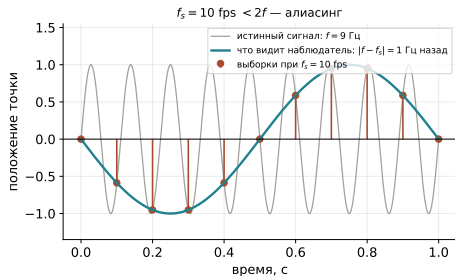
Пример. Колесо вращается со скоростью $f = 9$ Гц. Чтобы «видеть» его правильно, нужно $\text{fps} > 18$. При $\text{fps} = 10$ — увидим ложный сигнал, при $\text{fps} = 30$ — восстановим правильно (следующий слайд).

3. Временной алиасинг: колесо крутится назад

Определение: временной алиасинг

Ситуация, когда при нарушении условия Найквиста ($f_s < 2f$) быстрый сигнал в кадрах выглядит как медленный сигнал другой частоты (часто — в обратную сторону).

Временной алиасинг: что меняется при увеличении fps



3. Алиасинг: где встречается в жизни и в ML

Что видим глазами:

- ▶ Пропеллеры самолётов «стоят» или крутятся в обратную сторону
- ▶ Колёса машин при разгоне идут назад
- ▶ Мерцание LED-освещения (100 Гц) при 25 fps → стробоскоп

Почему это важно для нас:

- ▶ Быстрые действия (удар в теннисе, взмах руки) при низком fps теряют информацию
- ▶ Модель распознавания действий получит на вход *неверную* динамику
- ▶ Единственный способ побороть — снимать с достаточным fps на этапе записи; после — уже нельзя восстановить

Практика: для распознавания быстрых действий планируйте $\text{fps} \geq 60$.
Для «спокойного» видео (лица, документы, разговор) хватает 25–30.

4. Сколько весит несжатое видео

Определение: битрейт

Битрейт — количество бит в секунду, необходимое для передачи или хранения видео. Обозначение: Мбит/с (мегабит в секунду).

Битрейт *сырого* (несжатого) видео:

$$B_{\text{raw}} = W \cdot H \cdot C \cdot b \cdot \text{fps}$$

где W — ширина, H — высота, $C = 3$ канала, $b = 8$ бит на канал, fps — частота кадров.

Считаем для 1080p, 30 fps:

- ▶ 1 кадр: $1920 \cdot 1080 \cdot 3 \cdot 8 = 49,766,400$ бит ≈ 6.2 МБ
- ▶ 30 fps: ≈ 1.49 Гбит/с ≈ 187 МБ/с
- ▶ 1 час: ≈ 670 ГБ

Реальный файл 1080p H.264 весит ~ 5 Мбит/с. Значит, кодек сжимает $\approx$ **300 раз**. Как? Отвечаем в разделах 7–8.

5. Цвет: почему кодекам не хватает RGB

Мы привыкли к RGB (красный, зелёный, синий). Кодеки работают в другой системе — YCbCr. Зачем?

Ключевое наблюдение (физиология зрения):

- ▶ Глаз чувствителен к *яркости* — светлее/темнее
- ▶ К *цвету* чувствителен слабее в 2–4 раза

Если разделить сигнал на «яркость отдельно» и «цвет отдельно» — цвет можно хранить с меньшим разрешением. Это экономит место без заметной потери качества.

Определение: YCbCr

Цветовое пространство с тремя компонентами: Y — *luma*, яркость; C_b — синяя цветоразность (насколько «синее» яркости); C_r — красная цветоразность. Зелёная компонента восстанавливается из Y , C_b , C_r при декодировании.

5. RGB → YCbCr (стандарт BT.709)

Яркость (luma):

$$Y = 0.2126R + 0.7152G + 0.0722B$$

Веса подобраны под физиологию: глаз чувствительнее всего к зелёному, поэтому у G наибольший коэффициент.

Цветоразности (chroma):

$$C_b = 0.5389(B - Y), \quad C_r = 0.6350(R - Y)$$

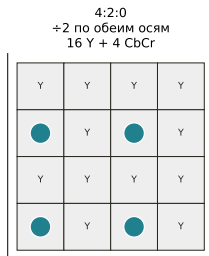
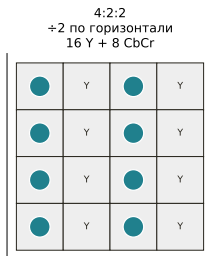
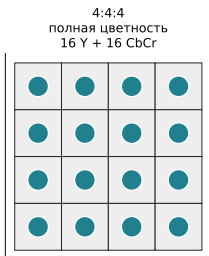
C_b, C_r лежат в диапазоне $[-0.5, 0.5]$. Нулевые $C_b = C_r = 0$ — серый.

Три стандарта матриц: BT.601 (SD), BT.709 (HD/1080p — по умолчанию), BT.2020 (UHD/HDR). Если перекодировать HD-файл с матрицей SD — цвета «поплывут». Всегда проверяем метаданные (`color_matrix` через `ffprobe`).

6. Chroma subsampling: экономим на цвете

Определение: chroma subsampling

Хранение компонент C_b, C_r с *меньшим* разрешением, чем яркость Y . Обозначение $J:a:b:J = 4$ — эталон 4 пикселя, a — сколько сэмплов хромы в верхней строке, b — в нижней.



4:4:4 — цвет как яркость (профессиональное производство). **4:2:2** — цвет в 2 раза грубее по горизонтали (broadcast). **4:2:0** — цвет в 2 раза грубее по обеим осям, экономия $\times 2$ без потери яркости. *Стандарт де-факто для всего потребительского видео: YouTube, телефоны, датасеты.*

6. Chroma 4:2:0 — практические следствия

Что становится хуже:

- ▶ Мелкие цветные детали «мажутся»: красный светодиод на чёрном фоне выглядит как розовое пятно 2×2
- ▶ Границы цветных объектов размываются на 1 пиксель
- ▶ Красный текст на белом фоне: буквы читаются (яркость), но цвет фона по краям «плывёт»

Что *не* становится хуже:

- ▶ Резкость самой картинки (яркость Y сохранена полностью)
- ▶ Разборчивость чёрного текста на белом фоне

Форматы пикселей в ffprobe: $yuv420p = 4:2:0$ 8-бит, $yuv422p = 4:2:2$, $yuv444p = 4:4:4$. **99% датасетов — это $yuv420p$.**

7. Сжатие видео: две избыточности, два механизма

Как из 1.49 Гбит/с получаются 5 Мбит/с? Кодеки эксплуатируют две избыточности:

Intra (внутрикадровая)

Соседние пиксели *в одном кадре* похожи. Небо гладкое, стена гладкая.

Механизм: блок $8 \times 8 \rightarrow$ DCT $\rightarrow$ квантование $\rightarrow$ энтропия. Как в JPEG.

Разберём каждый механизм отдельно.

Inter (межкадровая)

Соседние *кадры* почти совпадают. Меняется малая часть.

Механизм: находим сдвиг блока, кодируем вектор и остаток. Даёт *основную* часть сжатия.

7. Intra: шаг 1 — блочное разбиение

Определение: блок

Квадратный участок кадра фиксированного размера, обычно 8×8 пикселей (в JPEG всегда 8×8 ; в H.264 — 4×4 или 8×8 ; в HEVC — до 64×64).

Кадр разбивается на непересекающиеся блоки, каждый блок кодируется независимо.

Пример: кадр 1080p (1920×1080) при блоке 8×8 содержит

$$(1920/8) \times (1080/8) = 240 \times 135 = 32,400 \text{ блоков.}$$

Почему блоки, а не весь кадр целиком?

- ▶ DCT на весь кадр — очень дорого (N^4 операций)
- ▶ Локальная статистика: небо и лицо — разные, лучше жать по отдельности
- ▶ Ошибка сжатия ограничена блоком (не «размажется» по всему кадру)

7. Intra: шаг 2 — DCT переводит в частоты

Определение: дискретное косинусное преобразование (DCT)

Линейное преобразование, которое представляет блок $N \times N$ пикселей как взвешенную сумму N^2 базисных «шахматок» разной частоты.

Формула 2D-DCT для блока $N \times N$:

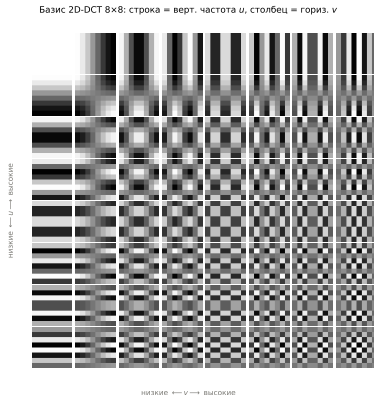
$$F(u, v) = \alpha(u) \alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos\left(\frac{(2x+1)u\pi}{2N}\right) \cos\left(\frac{(2y+1)v\pi}{2N}\right)$$

7. Intra: DCT — что означают символы

- ▶ $f(x, y)$ — значение пикселя в блоке (координаты $x, y \in \{0, \dots, N - 1\}$)
- ▶ $F(u, v)$ — DCT-коэффициент; u — вертикальная частота, v — горизонтальная
- ▶ $\alpha(0) = \sqrt{1/N}$, $\alpha(k) = \sqrt{2/N}$ при $k \geq 1$ — нормировка
- ▶ $F(0, 0)$ — среднее по блоку (DC-компонента); правый нижний угол — самые высокие частоты

Как понимать формулу: каждый коэффициент $F(u, v)$ — это «насколько сильно в блоке присутствует волна с частотой (u, v) ». Преобразование *обратимо и без потерь* — потери появятся только на следующем шаге (квантование).

7. Intra: как выглядит базис DCT 8×8



Любой блок 8×8 представим как взвешенная сумма этих 64 картинок с коэффициентами $F(u, v)$. Естественные изображения имеют почти всю энергию в верхнем левом углу (низкие частоты).

7. Intra: шаг 3a — что такое квантование

Определение: квантование

Каждый DCT-коэффициент $F(u, v)$ делится на своё число $Q(u, v)$ и округляется:

$$\hat{F}(u, v) = \text{round}\left(\frac{F(u, v)}{Q(u, v)}\right).$$

Это *единственное* место в кодеке, где теряется информация.

Зачем: высокие частоты глазу почти незаметны $\Rightarrow$ можно грубо округлить их до нуля, блок почти не изменится, а длинные нулевые последовательности потом отлично сжимаются энтропийным кодером (следующий шаг). Низкие частоты округляем точнее — глаз их видит.

7. Intra: шаг 36 — как выглядит матрица Q (JPEG, качество 50)

$$Q_{50} = \begin{pmatrix} \mathbf{16} & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & \mathbf{99} \end{pmatrix}$$

Структура: числа *растут* от верхнего-левого угла к правому-нижнему — высокие частоты делятся на большее число $\Rightarrow$ чаще зануляются.

$Q(0,0) = 16$ — DC (среднее блока): ошибка малая.

Пример: $F(0,0) = 400 \rightarrow 400/16 = 25$.

$Q(7,7) = 99$ — высокая частота: почти всегда ноль.

Пример: $F(7,7) = 40 \rightarrow 40/99 = 0.4 \rightarrow 0$.

7. Intra: сквозной пример — пиксели и DCT

Блок 8×8 темно-голубого неба (плавный градиент). Ниже — пиксели и их частотное представление:

1. Пиксели (яркость Y , 0-255):

$$\begin{pmatrix} 52 & 55 & 61 & 66 & 70 & 61 & 64 & 73 \\ 63 & 59 & 55 & 90 & 109 & 85 & 69 & 72 \\ 62 & 59 & 68 & 113 & 144 & 104 & 66 & 73 \\ 63 & 58 & 71 & 122 & 154 & 106 & 70 & 69 \\ 67 & 61 & 68 & 104 & 126 & 88 & 68 & 70 \\ 79 & 65 & 60 & 70 & 77 & 68 & 58 & 75 \\ 85 & 71 & 64 & 59 & 55 & 61 & 65 & 83 \\ 87 & 79 & 69 & 68 & 65 & 76 & 78 & 94 \end{pmatrix}$$

2. После DCT $F(u, v)$:

$$\begin{pmatrix} -415 & -30 & -61 & 27 & 56 & -20 & -2 & 0 \\ 4 & -22 & -61 & 10 & 13 & -7 & -9 & 5 \\ -47 & 7 & 77 & -25 & -29 & 10 & 5 & -6 \\ -49 & 12 & 34 & -15 & -10 & 6 & 2 & 2 \\ 12 & -7 & -13 & -4 & -2 & 2 & -3 & 3 \\ -8 & 3 & 2 & -6 & -2 & 1 & 4 & 2 \\ -1 & 0 & 0 & -2 & -1 & -3 & 4 & -1 \\ 0 & 0 & -1 & -4 & -1 & 0 & 1 & 2 \end{pmatrix}$$

Энергия сконцентрировалась в верхнем-левом углу ($F(0, 0) = -415$ — средняя яркость блока); высокие частоты в правом-нижнем углу — уже малые. Это типично для любых естественных блоков.

7. Intra: сквозной пример — после квантования

Каждый коэффициент делим на своё число из матрицы Q_{50} и округляем:

$$\hat{F}(u, v) = \text{round}(F(u, v)/Q_{50}(u, v)).$$

3. Квантованные коэффициенты $\hat{F}$:

$$\begin{pmatrix} -26 & -3 & -6 & 2 & 2 & -1 & 0 & 0 \\ 0 & -2 & -4 & 1 & 1 & 0 & 0 & 0 \\ -3 & 1 & 5 & -1 & -1 & 0 & 0 & 0 \\ -4 & 1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Примеры вычислений:

- $F(0, 0)/Q(0, 0) = -415/16 = -25.9 \rightarrow -26$
- $F(2, 2)/Q(2, 2) = 77/16 = 4.8 \rightarrow 5$
- $F(0, 3)/Q(0, 3) = 27/16 = 1.7 \rightarrow 2$
- $F(5, 5)/Q(5, 5) = 1/104 = 0.01 \rightarrow 0$

Итог: было 64 коэффициента, из них ненулевых было 32; после квантования осталось только 14. Остальные 50 — нули, готовые для RLE.

7. Intra: шаг 4 — энтропия (продолжаем тот же пример)

4а. Зигзаг из матрицы $\hat{F}$ — читаем коэффициенты от DC к высоким частотам по диагоналям:

$$\underbrace{-26}_{\text{DC}}, -3, 0, -3, -2, -6, 2, -4, 1, -4, 1, 1, 5, 1, 2, -1, -1, \underbrace{0, 0, \dots, 0}_{47 \text{ нулей}}$$

4б. Run-Length Encoding — кодируем парами (число нулей перед, значение):

$$(0, -26), (0, -3), (1, -3), (0, -2), (0, -6), (0, 2), (0, -4), \dots, (0, -1), \text{EOB}$$

Символ **EOB** (End Of Block) заменяет *все 47 нулей* в конце одним токеном.

4в. Энтропийный кодер (Huffman / CABAC / ANS): частым токенам — короткие коды, редким — длинные. EOB встречается почти всегда → код длины 2-3 бита.

Итог: 64 пикселя $\times$ 8 бит = 512 бит $\rightarrow \sim 80$ -100 бит после всей цепочки — сжатие $\times 5$ - $\times 6$.

8. Inter: компенсация движения

Определение: компенсация движения (motion compensation)

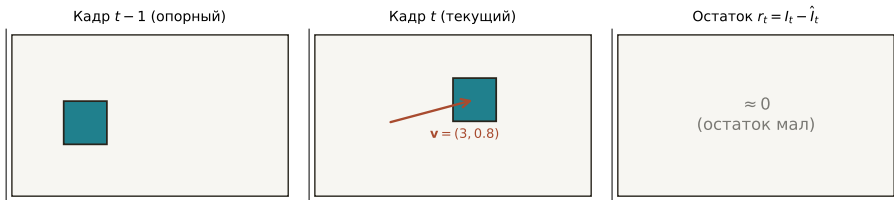
Способ предсказать кадр I_t сдвигом блоков предыдущего кадра I_{t-1} .
Кодируется не сам кадр, а вектор сдвига $\mathbf{v}$ и малая разность.

Предсказание кадра:

$$\hat{I}_t(p) = I_{t-1}(p + \mathbf{v}(p))$$

где $p = (x, y)$ — координата блока в кадре t ; $\mathbf{v}(p) = (v_x, v_y)$ — вектор движения этого блока; $\hat{I}_t$ — предсказанный кадр (сдвинутый I_{t-1}).

Компенсация движения: кодируется вектор $\mathbf{v}$ и почти нулевой остаток



8. Inter: SAD — как измерить «похожесть» блоков

Определение: SAD (Sum of Absolute Differences)

Мера различия между блоком в кадре t и блоком, сдвинутым на $\mathbf{v}$ в опорном кадре $t - 1$:

$$\text{SAD}(\mathbf{v}) = \sum_{p \in \text{block}} |I_t(p) - I_{t-1}(p + \mathbf{v})|.$$

Сумма модулей попиксельной разности. Чем меньше — тем лучше блок совпал. Поиск лучшего вектора: $\mathbf{v}^* = \arg \min_{\mathbf{v} \in W} \text{SAD}(\mathbf{v})$.

Численный пример — блок 4×4 (для наглядности; в реальности 16×16):

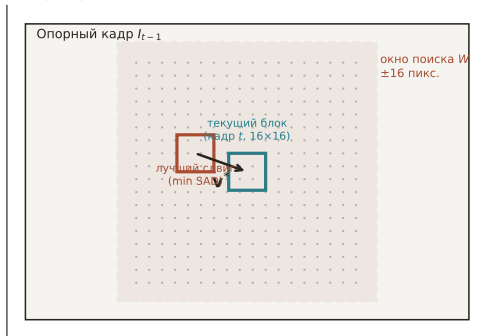
I_t (текущий):	$I_{t-1}(p + \mathbf{v})$ (кандидат):	$ I_t - I_{t-1} $:
$\begin{pmatrix} 50 & 55 & 60 & 62 \\ 52 & 58 & 61 & 63 \\ 55 & 60 & 62 & 64 \\ 58 & 62 & 64 & 65 \end{pmatrix}$	$\begin{pmatrix} 51 & 54 & 59 & 63 \\ 50 & 59 & 60 & 64 \\ 56 & 61 & 61 & 65 \\ 57 & 63 & 65 & 65 \end{pmatrix}$	$\begin{pmatrix} 1 & 1 & 1 & 1 \\ 2 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$

$\text{SAD} = 1 + 1 + \dots + 0 = \mathbf{17}$ — очень мало, блоки почти совпали. Для случайного сдвига было бы $\text{SAD} \sim 500\text{--}2000$.

8. Inter: полный перебор — простой, но дорогой

Идея: перебрать все возможные сдвиги в окне W вокруг позиции блока и выбрать сдвиг с минимальным SAD.

Полный перебор: в окне W ($33 \times 33 = 1089$ вариантов) ищем сдвиг с min SAD



Окно W : ± 16 пикселей по каждой оси $\Rightarrow 33 \times 33 = \mathbf{1089}$ кандидатов на блок.

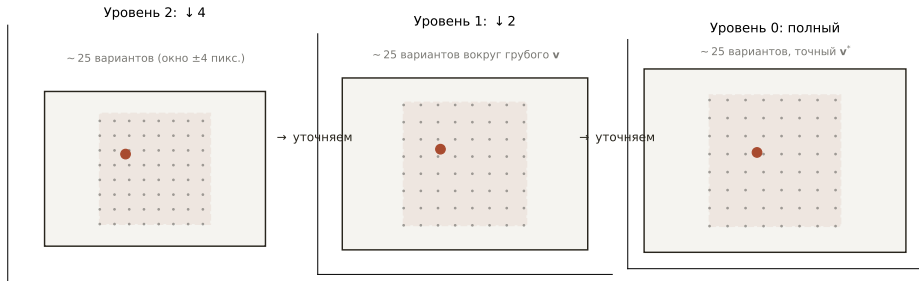
Стоимость. 1080p: ~ 8100 блоков $\times$ 1089 кандидатов $\times$ 256 сложений (SAD блока 16×16) $\approx \mathbf{2 \text{ млрд операций / кадр}}$. При 30 fps ~ 60 GOPS — неподъёмно для CPU реального времени.

Проблема: большинство кандидатов заведомо плохие. Нужен способ их не проверять.

8. Inter: иерархический поиск — грубо, потом точно

Идея: строим пирамиду уменьшенных копий. Грубый поиск на маленькой копии → уточнение на большей.

Иерархический поиск: грубый → точный (вместо 1089 — ~ 75 сравнений)



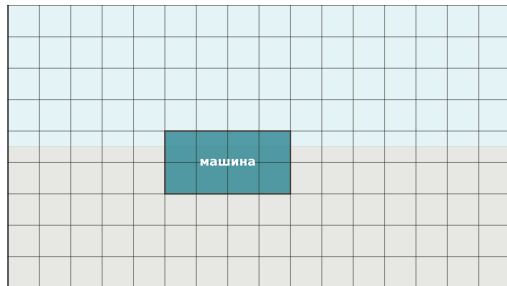
- **Ур. 2** (↓ 4, 480×270): окно ± 4 , ~ 25 вариантов → грубый $\mathbf{v}_2$.
- **Ур. 1** (↓ 2, 960×540): окно ± 2 вокруг $2\mathbf{v}_2$, ~ 25 сравнений → $\mathbf{v}_1$.
- **Ур. 0** (полный, 1920×1080): окно ± 2 вокруг $2\mathbf{v}_1$, ~ 25 → точный $\mathbf{v}^*$.

Итог: 1089 сравнений → ~ 75, ускорение $\sim 15 \times$. Subpel-уточнение до $1/4$ пикселя даёт +20–30% сжатия на плавных объектах.

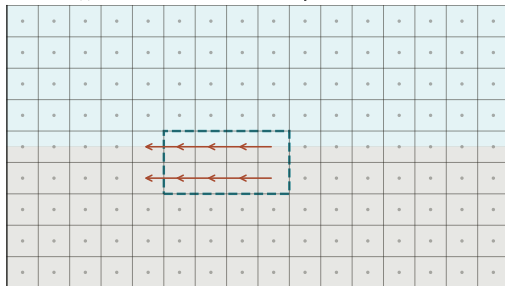
8. Inter: весь кадр — поле векторов

Мы разобрали поиск вектора для *одного* блока. На всем кадре процедура повторяется для каждого блока независимо.

1. Кадр t разбивается на сетку блоков (16×16 пикселей)



2. Для каждого блока свой $\mathbf{v}$ — получаем поле движения



• статичный блок ($\mathbf{v} = 0$)

→ блок с ненулевым $\mathbf{v}$ (объект движется)

1. Кадр 1080p режется на сетку $120 \times 68 \approx 8100$ блоков 16×16 .
2. Для каждого блока — свой поиск в окне ± 16 вокруг его позиции (то, что разобрали). На выходе — свой $\mathbf{v}_i^*$.
3. Статичные области (фон, небо) $\rightarrow \mathbf{v} \approx 0$; движущиеся объекты (машина) $\rightarrow$ векторы, указывающие откуда *ВЗЯТЬ* пиксели в опорном кадре.

8. Inter: как векторы не съедают выигрыш — предсказание вектора

Проблема: 8100 блоков $\times$ 10 бит/вектор = 81 000 бит только на векторы — слишком много.

Идея: соседние блоки движутся похоже (объект цельный) $\Rightarrow$ вектор можно *предсказать* по соседям:

$$\hat{\mathbf{v}}_i = \text{median}(\mathbf{v}_{\text{лев}}, \mathbf{v}_{\text{верх}}, \mathbf{v}_{\text{верх-прав}}), \quad \Delta \mathbf{v}_i = \mathbf{v}_i - \hat{\mathbf{v}}_i.$$

В поток уходит $\Delta \mathbf{v}_i$, а не сам $\mathbf{v}_i$.

- ▶ **Фон:** соседи $\mathbf{v} = 0 \Rightarrow \Delta \mathbf{v} = 0 \Rightarrow$ **0 бит** (флаг «skip»).
- ▶ **Машина:** соседи $\mathbf{v} \approx (-3, 0)$, шум $\pm 1 \Rightarrow$ **2-4 бита**.
- ▶ **Граница объекта:** соседи врут $\Rightarrow$ **полные 10 бит** (но таких блоков $\sim 5\%$).

Среднее по кадру: ~ 2 бита на вектор вместо 10 — экономия $5\times$ без потерь точности.

8. Inter: что кодируется

Шаг 4. Что уходит в поток. Для каждого блока кодек записывает:

- ▶ вектор $\mathbf{v}^*$ — обычно ~ 10 бит (часто ещё короче: соседние блоки двигаются похоже $\rightarrow$ кодируется *разность* векторов);
- ▶ остаток $r_t(p) = I_t(p) - I_{t-1}(p + \mathbf{v}^*)$ — прогоняется через *тот же intra-конвейер* (DCT $\rightarrow$ Q $\rightarrow$ энтропия).

Если движение объяснило почти всё, $r_t \approx 0$ и весь блок стоит всего **20-50 бит** вместо 80-100 для intra. Отсюда второй вклад в сжатие — ещё $\times 3$ - $\times 5$ к intra.

9. От одного кадра к потоку: типы кадров

Мы разобрали, как один кадр предсказывается из соседнего (inter) и как кадр кодируется независимо (intra). Как это собирается в поток? Кодек чередует три типа кадров:

Три типа кадров

I (Intra): полное intra-кодирование; декодируется *независимо*; самый тяжёлый, точка входа для seek.

P (Predicted): inter-ссылки на *предыдущие* I/P; векторы + остаток; в 5–10 раз легче I.

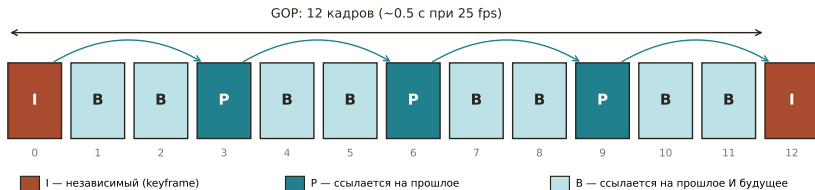
B (Bidirectional): inter-ссылки на *предыдущие И будущие*: блок от стены — из предыдущего, блок от нового объекта — из следующего; самый лёгкий.

9. GOP — группа кадров

GOP (Group of Pictures)

Последовательность кадров от одного I-кадра до следующего. Обычно 0.5–10 секунд (12–250 кадров).

Короткий GOP — быстрый seek и быстрая навигация, но в 1.5–2 раза больший размер файла. Длинный GOP — хорошее сжатие, но медленный seek.



На примере: GOP = 12 кадров, структура IBVBPVBPVBPVBI. Стрелки не показаны, но P-кадры ссылаются на левый I/P, B-кадры — на соседей слева и справа.

9. Порядок декодирования $\neq$ порядок показа

Пример потока $I_0 B_1 B_2 P_3$: показывается в порядке 0, 1, 2, 3; декодируется в порядке 0, 3, 1, 2 (потому что B_1, B_2 зависят от P_3).

Почему seek дорогой: чтобы получить кадр k , декодер (1) ищет ближайший *предыдущий* I-кадр и (2) декодирует всё от него до k . При $GOP = 250$ (10 сек при 25 fps) в среднем декодируется 125 «лишних» кадров на каждый $seek(k)$.

Совет для ML: для случайного сэмплирования клипов либо перекодируйте с коротким GOP (-g 25), либо заранее нарежьте видео на 1-2 секундные фрагменты.

9'. Декодирование: весь конвейер в обратную сторону

Декодер выполняет все шаги кодера в обратном порядке:

Кодер (было)	Шаг декодера	Что получаем
энтропия (RLE Huffman/CABAC)	+ 1. энтропийное декодирование	коэф. $\hat{F}(u, v)$, векторы $\mathbf{v}$, типы кадров
квантование $\hat{F}$ $\text{round}(F/Q)$	= 2. деквантование $\tilde{F} = \hat{F} \cdot Q$	приближенные DCT-коэф. $\tilde{F}$ (F точно восстановить нельзя)
DCT $F = \text{DCT}(B)$	3. обратное DCT $\tilde{B} = \text{DCT}^{-1}(\tilde{F})$	блок пикселей (residual для P/B, кадр для I)
inter: $R = B - B_{\text{pred}}$	4. компенсация движения $\tilde{B} = \tilde{R} + \tilde{B}_{\text{pred}}(\mathbf{v})$	восстановленный блок в YCbCr
chroma subsampling 4:2:0	5. up-sampling хромы — интерполяция C_b, C_r	полные плоскости Y, C_b, C_r
RGB $\rightarrow$ YCbCr (BT.709)	6. YCbCr $\rightarrow$ RGB — обратная линейная формула	кадр RGB, готовый к показу

Шаги **1-3** — внутри блока, шаг **4** — межкадровый, шаги **5-6** — на уровне кадра. Порядок кадров восстанавливается из PTS/DTS.

9'. Декодирование: почему это важно для ML

Кодирование асимметрично. Motion estimation (поиск $\mathbf{v}^*$) — самый дорогой шаг кодера. В декодере этого шага нет — векторы уже в потоке, остаётся лишь применить их к референсным кадрам. Поэтому декодер в 10–100 раз быстрее кодера.

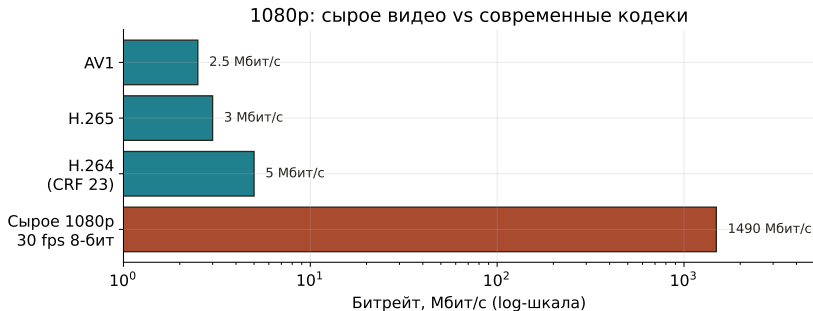
Следствия для ML-пайплайнов:

- ▶ Чтобы получить один кадр, надо выполнить шаги 1–6 для всех кадров от предыдущего I-кадра — отсюда стоимость seek из предыдущего слайда.
- ▶ Формат на выходе декодера — обычно YCbCr 4:2:0 (шаг 6 опционален). Конвертация в RGB — лишние $\sim 10\%$ CPU, её часто переносят на GPU или вообще не делают, если модель умеет читать YCbCr.
- ▶ **Теряемая информация** — только на шагах **2** (квантование) и **5** (up-sampling хромы). Все остальные шаги точно обратимы.
- ▶ ffmpeg и PyAV из коробки выполняют всю цепочку; NVIDIA NVDEC / Intel QSV делают шаги 1–5 на видеокарте — в $\sim 5\text{--}10\times$ быстрее CPU.

10. Битрейт: определение и порядок величин

Определение: качество и битрейт

Большой битрейт $\Rightarrow$ больше информации $\Rightarrow$ выше качество (при том же кодеке). Задача кодека — при заданном битрейте максимизировать воспринимаемое качество.



Разница $\times 300$ между сырым и H.264 — это работа intra + inter. Разница между кодеками — эффективность алгоритма при том же качестве.

10. Основные кодеки

Определение: кодек

Алгоритм сжатия и распаковки видеопотока. От англ. *coder* + *decoder*.

Мы разобрали H.264/AVC — блоки до 16×16 , DCT $4 \times 4/8 \times 8$, CAVLC/CABAC-энтропия, 1/4-pixel motion, I/P/B. Остальные кодеки в таблице — вариации на ту же тему:

Кодек	Год	vs H.264	Отличие от H.264
MJPEG	1990-е	$\times 0.1$	<i>Нет inter</i> : каждый кадр — JPEG. Плохое сжатие, зато мгновенный seek.
H.264/AVC	2003	$\times 1$	Кодек из нашей лекции — база для сравнения.
H.265/HEVC	2013	$\times 1.5-2$	Блоки до 64×64 (CTU) вместо 16, 35 intra-направлений вместо 9, только CABAC.
VP9	2013	$\times 1.5$	Открытый аналог HEVC от Google, без патентных платежей.
AV1	2018	$\times 2-2.5$	Блоки до 128×128 , warped motion, несколько опорных кадров, ANS вместо CABAC.
H.266/VVC	2020	$\times 2.5-3$	Гибкое разбиение, affine motion.

10. Контейнер $\neq$ кодек

Определение: контейнер

«Коробка» с потоками: видео, аудио, субтитры, метаданные. Форматы: MP4, MKV, WebM, MOV, AVI.

Один и тот же кодек может жить в разных контейнерах — одинаковые биты видеопотока, разная упаковка:

Файл	Что внутри
movie.mp4	кодек H.264 + аудио AAC в контейнере MP4
movie.mkv	тот же H.264 + аудио FLAC в контейнере MKV
movie.webm	кодек VP9 + аудио Opus в контейнере WebM

Расширение файла говорит только о контейнере. Чтобы узнать кодек — `ffprobe file.mp4`.

Итог лекции

1. Видео — дискретизация $I(x, y, t)$; fps ограничен теоремой Найквиста
2. Временной алиасинг возникает при $f_s < 2f$ и *необратим*
3. Сырое 1080p 30fps = 1.49 Гбит/с; кодек жмёт в ~ 300 раз
4. Кодеки работают в YCbCr 4:2:0 — экономия $\times 2$ на цвете
5. Intra: DCT + квантование + энтропия. Потери — только на квантовании
6. Inter: motion compensation. Кодировается вектор $\mathbf{v}$
7. I/P/B и GOP определяют стоимость случайного доступа
8. Стандарт де-факто — H.264 в MP4-контейнере

Литература

- ▶ **Richardson I.** — *The H.264 Advanced Video Compression Standard*
- ▶ **Sze, Budagavi, Sullivan** — *High Efficiency Video Coding (HEVC): Algorithms and Architectures*
- ▶ **Poynton C.** — *Digital Video and HD: Algorithms and Interfaces* — цвет, гамма, YCbCr, диапазоны
- ▶ **Wu C.-Y. et al.** — *Compressed Video Action Recognition*, CVPR 2018 — motion vectors как вход модели
- ▶ **FFmpeg documentation** — <https://ffmpeg.org/documentation.html>
- ▶ **Википедия: Discrete cosine transform, Chroma subsampling** — короткие обзорные статьи